Reinforcement Learning Summative Assignment Report

Student Name: Sharif Kiviiri

Video Recording: [Link to your Video 3 minutes max, Camera On, Share the entire Screen]
GitHub Repository: https://github.com/Sharif2138/Sharif_Kiviiri_rl_summative

1. Project Overview

This project trains a reinforcement learning agent to manage driver fatigue in a commercial fleet setting. The agent plays the role of an in-cab monitoring system that watches signs of drowsiness in a driver , eye closure, yawning, head movement and how long the driver has been on the road , and decides when to step in. The goal is to reduce the risk of fatigue-related accidents without triggering unnecessary or annoying alerts. I built a custom Gymnasium environment called DriverFatigueEnv that simulates this shift in 15-minute steps, and I trained four algorithms on it , DQN, REINFORCE, PPO and A2C using Stable-Baselines3 (with REINFORCE implemented from scratch in PyTorch). The environment is rendered in 3D using PyGame and OpenGL so the agent's decisions can actually be watched during a live simulation.

2. Environment Description

a. Agent(s)

The agent represents an automated in-cab safety system for a commercial driver (e.g. a bus). It does not control the vehicle itself , it only decides how to intervene when it detects signs of fatigue. At every 15-minute checkpoint it chooses one of four possible interventions, ranging from doing nothing to escalating the issue to a fleet manager.

b. Action Space

The action space is discrete with 4 actions:

- **Action 0 – Monitor only:** no intervention; used when the driver still looks alert.
- **Action 1 – Mild seat vibration:** a gentle physical nudge delivered through the seat to wake the driver up slightly.
- **Action 2 – In-cab alarm and red warning lights:** a stronger, audible and visual alert inside the cabin.
- **Action 3 – Critical alert to fleet manager:** the system escalates the situation outside the vehicle, notifying a manager that the driver may need to stop.

Each of these maps directly onto real hardware or software that already exists in fleet vehicles (haptic seat actuators, dashboard alarms, and a fleet management notification system), so the action space isn't abstract , it reflects tools a transport company could realistically install.

c. Observation Space

The agent observes a 4-dimensional continuous state, Box(low=[0, 0, -90, 0], high=[1, 10, 90, 1440]), of type float32:

Observation	Description	Source (Sensor/Camera/API/Dataset)	Encoding / Data Type	Range
PERCLOS	Percentage of time the eyes are closed, a standard drowsiness indicator	In-cab camera with eye-tracking	Float32	0.0 – 1.0
Yawn frequency	Number of yawns detected per monitoring window	In-cab camera with facial recognition	Float32	0.0 – 10 .0
Head pose angle	Pitch of the driver's head (drooping forward signals fatigue)	In-cab camera with head-pose estimation	Float32	-90° to 90°
Continuous drive time	Minutes driven without a break	Vehicle telematics / trip-logging API	Float32	0 – 1440 minutes

d. Reward Structure

The reward balances two things: penalising fatigue itself, and penalising interventions that aren't needed (since waking up an already-alert driver, or escalating too aggressively, has its own cost — driver annoyance, false alarms, lost trust in the system).

At every step, a baseline penalty is applied proportional to fatigue level:

reward += (10 − (PERCLOS × 25 + yawn × 1.5))

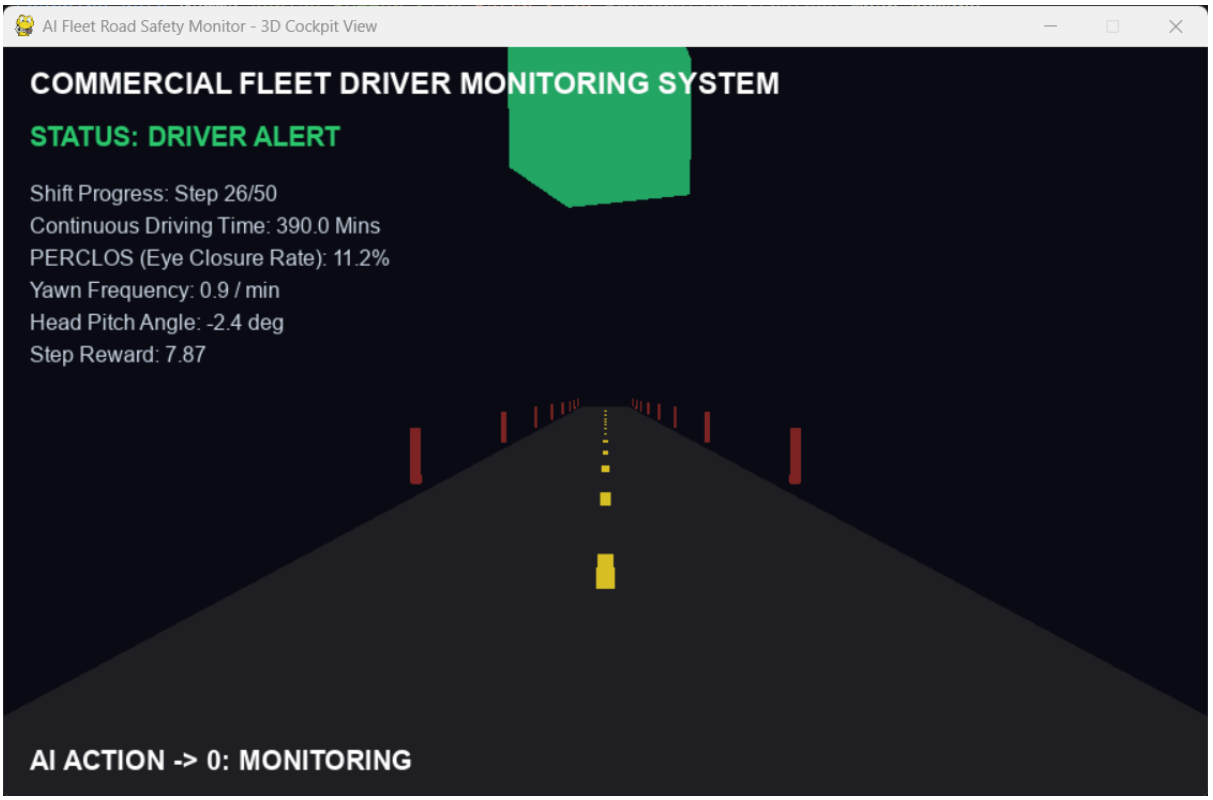
On top of that:

- Action 0 gives a small bonus (+2) only if the driver is genuinely still alert (PERCLOS < 0.3).
- Action 1 costs −1, and reduces PERCLOS by 0.15.
- Action 2 costs −10 if used too early (PERCLOS < 0.4, meaning it was unnecessary) but gives +5 if PERCLOS was actually high, rewarding correctly-timed strong intervention.
- Action 3 always costs −15, since escalating to a manager is a big deal and should be reserved for genuine emergencies.
- If PERCLOS ever exceeds 0.85 (the driver is dangerously close to falling asleep), the episode ends

immediately with a −100 penalty , this is the "near-miss" failure state the agent must learn to avoid.

An episode also ends after 50 steps (12.5 hours of simulated driving) if nothing critical happens.

Figure 1: 3D cockpit rendering of the DriverFatigueEnv, showing the road view, fatigue gauges, and live telemetry HUD.



3. System Analysis And Design

a. Deep Q-Network (DQN)

I used Stable-Baselines3's DQN with the default MlpPolicy (two fully-connected hidden layers, ReLU activations) to output Q-values over the 4 discrete actions. It uses the standard DQN mechanics: a replay buffer to store and resample past transitions, a target network that is updated more slowly than the main network to keep training stable, and epsilon-greedy exploration controlled by the exploration_fraction parameter. No changes were made to the core algorithm , the tuning was done entirely through the hyperparameters listed below.

b. Policy Gradient Method (REINFORCE, PPO, A2C)

REINFORCE was implemented manually in PyTorch: a small feed-forward network (2 hidden layers, ReLU, softmax output) produces action probabilities directly, and full-episode Monte Carlo returns are computed and normalised (mean-subtracted, std-divided) before being used in the policy-gradient update. There is no critic or baseline network . This is the plain, high-variance version of the algorithm.

PPO and A2C both use Stable-Baselines3's default MlpPolicy actor-critic networks. PPO adds a clipped surrogate objective so that policy updates don't move too far from the previous policy in a single step, which should make it more stable than plain policy gradients. A2C updates more frequently, using short rollouts (n_steps) and a synchronous advantage estimate instead of PPO's clipping mechanism.

4. Implementation

Each algorithm was tuned across 10 configurations, trained on the same environment for 50,000 timesteps per run, and evaluated over 10 fresh episodes with a deterministic policy.

a. DQN

Experiment	Learning Rate	Gamma	Replay Buffer Size	Batch Size	Exploration Strategy	Mean Reward
1	0.001	0.99	10000	32	0.1	**352.19**
2	0.0001	0.99	10000	32	0.1	301.74
3	0.0005	0.95	50000	64	0.2	321.77
4	0.001	0.95	50000	64	0.1	340.96
5	0.0001	0.90	20000	128	0.05	287.46
6	0.0005	0.99	10000	128	0.15	320.44
7	0.001	0.9	50000	32	0.2	330.05
8	0.0001	0.99	100000	64	0.1	274.95
9	0.0005	0.95	20000	32	0.05	315.76
10	0.001	0.99	100000	128	0.2	345.69

b. REINFORCE

Run	Learning Rate	Gamma	Hidden Layer Size	Mean Reward
1	0.001	0.99	64	-305.95
2	0.0005	0.99	64	-451.29
3	0.001	0.95	128	-100.11
4	0.0005	0.95	128	-453.33
5	0.0001	0.99	64	**81.72**

6	0.001	0.90	64	-454.95
7	0.0005	0.90	128	-432.36
8	0.0001	0.95	64	-442.57
9	0.002	0.99	32	-93.83
10	0.0005	0.99	32	-423.32

c. PPO

Run	Learning Rate	Gamma	N-Steps	Batch Size	Mean Reward
1	0.0003	0.99	128	32	264.80
2	0.001	0.99	256	64	286.84
3	0.0001	0.95	128	32	237.03
4	0.0005	0.99	512	128	147.28
5	0.0003	0.90	128	32	335.18
6	0.001	0.95	256	64	285.19
7	0.0005	0.99	128	64	-436.77
8	0.0001	0.99	256	32	-291.34
9	0.0003	0.95	512	64	333.36
10	0.0005	0.90	256	128	**335.29**

d. A2C

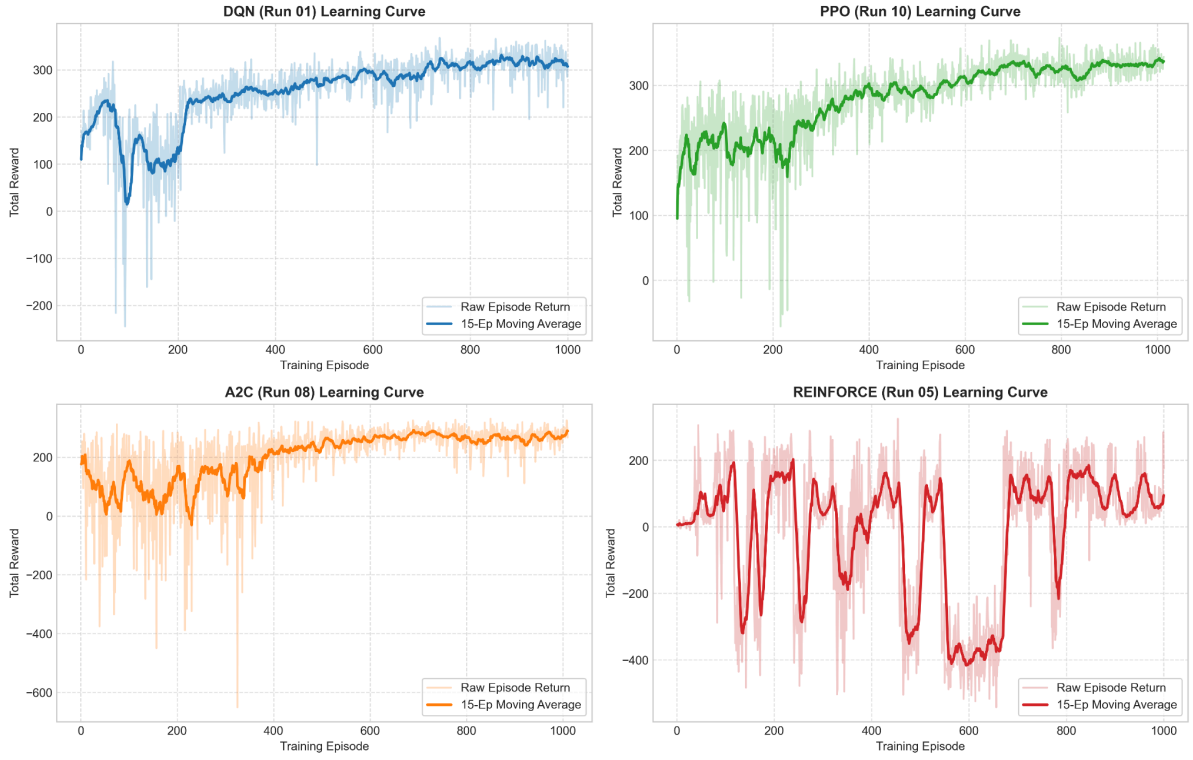
Run	Learning Rate	Gamma	N-Steps	Mean Reward
1	0.0007	0.99	5	67.28
2	0.001	0.99	10	83.99
3	0.0003	0.95	5	273.81
4	0.0005	0.99	20	- 411.73
5	0.0007	0.90	5	- 414.59
6	0.001	0.95	10	-423.16
7	0.0003	0.99	20	-398.98

8	0.0005	0.95	5	**290.77**
9	0.0007	0.99	10	17.18
10	0.0001	0.99	5	-413.63

Across all four algorithms, a clear pattern shows up: DQN is remarkably consistent . Every single one of its 10 runs landed in a fairly narrow positive band (275 to 352), regardless of which hyperparameters were used. The three policy-based methods behave very differently. Their best runs are competitive with DQN (PPO's best run reached 335, A2C's reached 291), but several of their configurations collapsed into deeply negative territory (below −400), meaning the policy essentially learned to trigger the terminal failure state repeatedly instead of avoiding it. This tells me the environment's sparse, sharply negative terminal penalty (−100 for a near-miss, on top of the ongoing fatigue penalty) creates a difficult, high-variance landscape for gradient-based policy methods, especially when gamma is high (0.99) and the policy has less "value-based grounding" to fall back on. Lower gamma values (0.90 – 0.95) and moderate learning rates tended to give both PPO and A2C their strongest results, since the agent doesn't chase distant rewards as aggressively and updates more conservatively.

5. Results Discussion

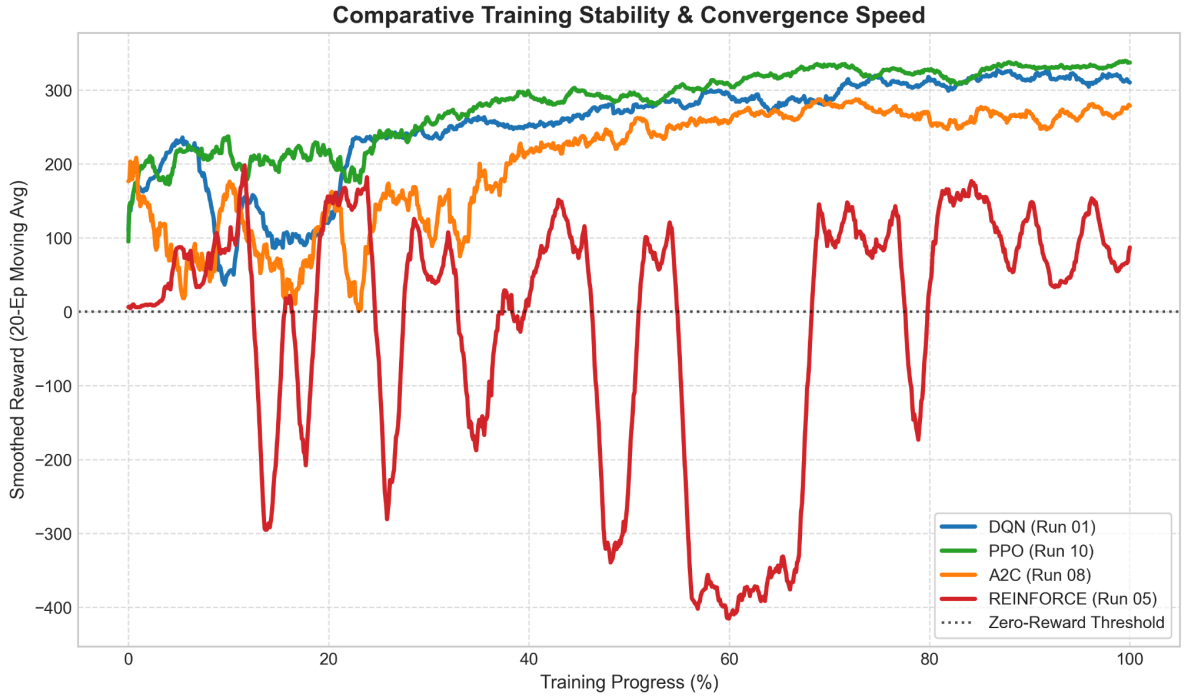
a. Cumulative Rewards

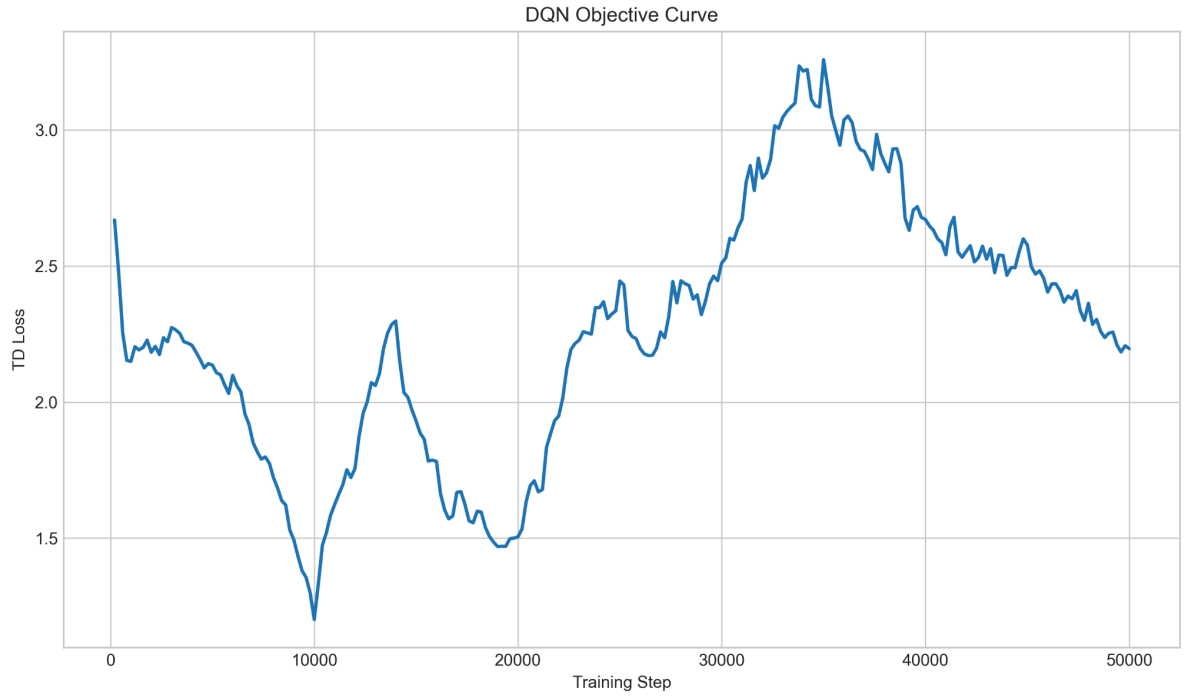


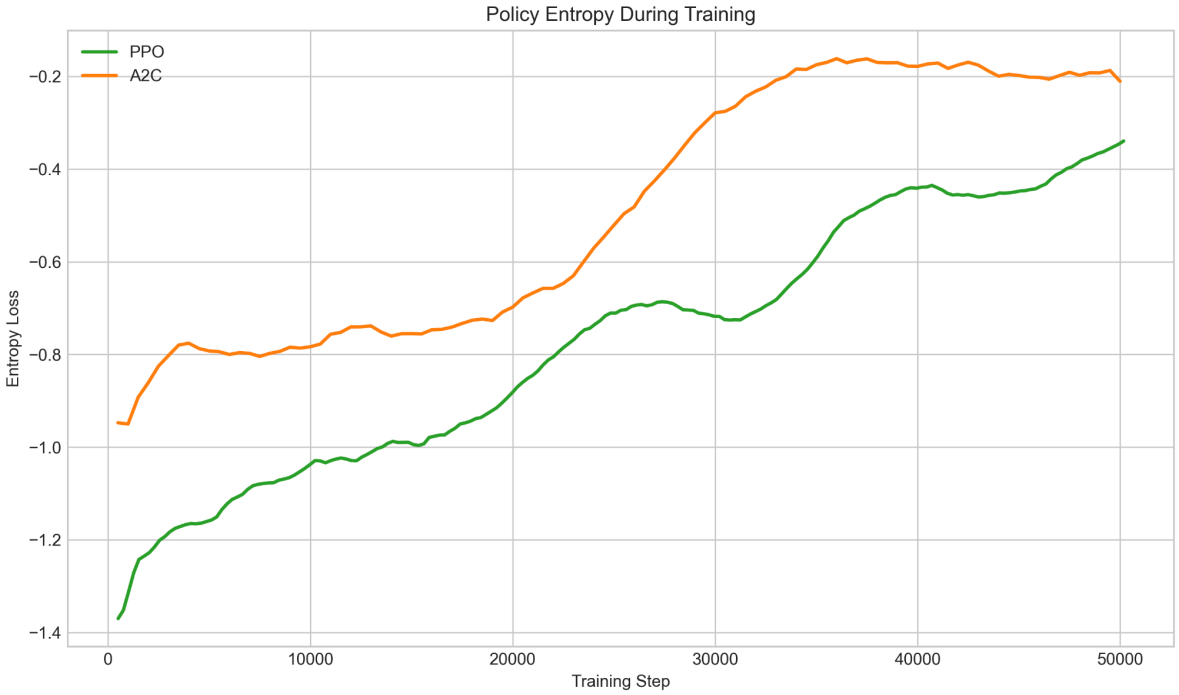
The subplot grid (cumulative_rewards_subplots.png) shows the raw and smoothed learning curves of the best run for each algorithm. DQN's curve climbs steadily and settles into a stable plateau with relatively low noise around the moving average , consistent with its narrow spread of final rewards across all 10 configurations. PPO's curve is noisier and takes longer to settle, with visible dips before it recovers, reflecting the on-policy updates reacting to the volatile early episodes. A2C's curve is the most jagged of the three SB3 methods, which fits with A2C's short rollout updates (5 steps) making each gradient step noisier. REINFORCE's curve is the

most erratic overall , long stretches of negative reward with sudden brief improvements, which lines up with the fact that 8 out of its 10 hyperparameter configurations ended up deeply negative. This is expected behaviour for a plain REINFORCE implementation with no critic to reduce variance. .

Training Stability





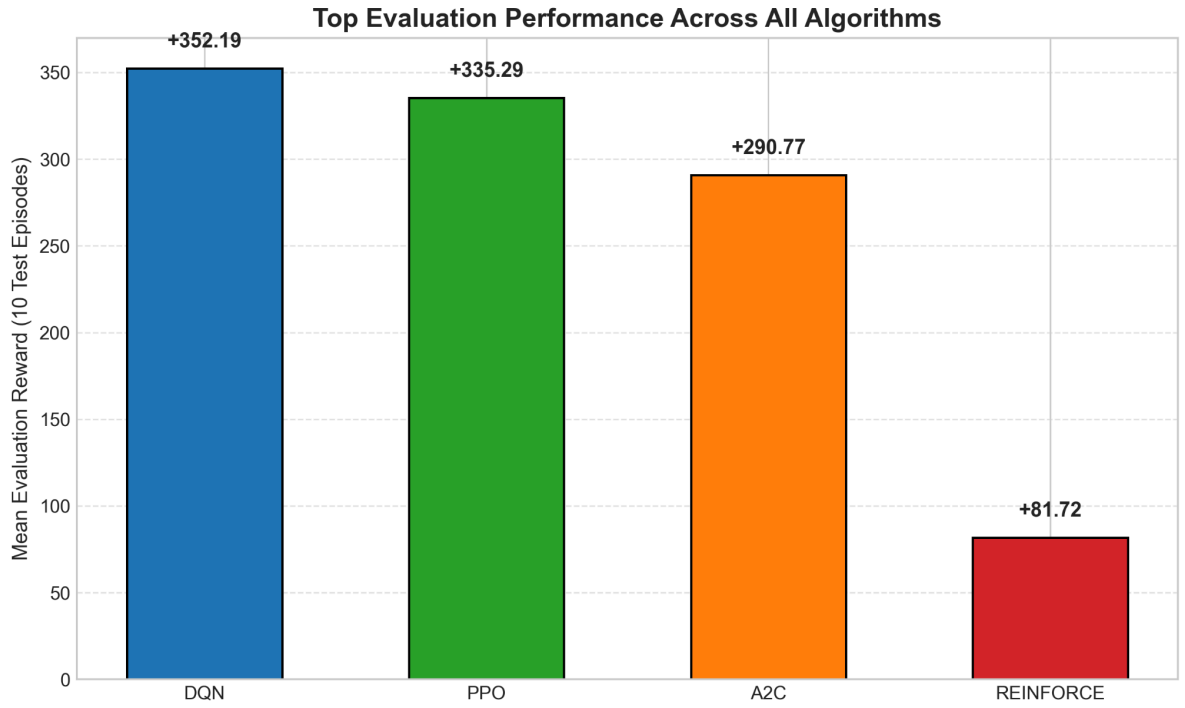


b. Episodes To Converge

Looking at the training stability plot (training_stability.png), DQN converges the fastest and reaches a stable positive reward earliest in training, thanks to experience replay letting it reuse transitions many times rather than depending on fresh trajectories. PPO takes noticeably longer to settle, generally needing well over half of the training budget before its curve flattens. A2C's smoothed reward oscillates around zero for a long stretch before climbing, showing slower and less certain convergence. REINFORCE barely converges within the given training budget for most runs . Its best-performing configuration only pulled ahead late in training, which suggests that with more episodes or a variance-reduction technique (like a value baseline), it might do better, but as implemented it's the slowest and least reliable to converge.

(see Figure 3, Section 5a)

c. Generalization



The leaderboard plot (generalization_test.png) compares each algorithm's best evaluation score on 10 fresh, unseen driving-shift episodes. DQN's best model (Run 1, reward 352.19) generalises best overall, followed closely by PPO (335.29) and A2C (290.77), with REINFORCE trailing well behind at 81.72. This gap makes sense: DQN's value-based approach directly estimates the expected return of each action in each state, so it handles new random fatigue trajectories more robustly, whereas REINFORCE's policy was only ever trained on a small number of full-episode samples per update and never developed as reliable a mapping between states and correct interventions.

The DQN objective curve (dqn_objective_curve.png) shows the TD loss decreasing and stabilising over training, which supports the idea that the Q-network is converging toward consistent value estimates rather than oscillating. The policy entropy plot (policy_entropy.png) for PPO and A2C shows entropy loss decreasing over time for both algorithms, meaning both policies become more confident (less random) in their action choices as training goes on . PPO's entropy decreases more smoothly, while A2C's is a bit more jagged, again reflecting its noisier, shorter-horizon updates.

6. Conclusion and Discussion

DQN was the best-performing and most reliable algorithm for this environment, achieving both the highest mean reward (352.19) and the most consistent results across all 10 hyperparameter combinations. This makes sense given the nature of the task: the environment has a discrete, small action space and a state space where the "correct" action mostly depends on the current fatigue readings rather than on long, complex sequences of decisions , exactly the kind of setting where value-based methods tend to do well. PPO came a reasonably close second and is likely the best choice among the policy-gradient family if a policy-based approach is preferred, since its clipped updates gave it more stability than A2C or REINFORCE. A2C performed decently in its best configuration but was very sensitive to the choice of gamma and rollout length. REINFORCE was the weakest and least stable method here, which is expected . Without a baseline or critic, its updates have high variance, and

the environment's sharp terminal penalty punishes that instability heavily.

If I had more time, I would try adding a value-based baseline to the REINFORCE implementation to reduce variance, and I would experiment with reward shaping , for example, softening the terminal penalty slightly to see whether it helps PPO and A2C avoid the catastrophic collapses seen in some runs. I would also like to test the best DQN model more rigorously against edge-case starting states (e.g. a driver already fatigued at the start of the shift) to check that its policy holds up outside the "average" conditions it was trained on.